Semantic Compiler: Safe LLM Orchestration of Neural Network Topology via Typed DSL

A Design Pattern for Physics-Constrained Networks, Demonstrated on Real-Time Option Pricing

OpKAN Engineering Team
`github.com/RicardoLaMo/OpKAN`

March 2026

Abstract

Giving a large language model real-time structural control over a running neural network creates a fundamental interface problem: the LLM produces unstructured text; the network demands typed, mathematically consistent structural updates. Naive approaches — evaluating LLM-generated code at runtime — introduce code injection vulnerabilities and leave the model in partially-mutated state on failure. We present the **semantic compiler** design pattern: the LLM's output is a typed, Pydantic-validated domain-specific language (DSL) object; formula strings are validated by static AST analysis before storage; and mutations are applied atomically with `deepcopy` rollback on failure. These three properties make every topology intervention type-safe, vocabulary-restricted, and transactionally reversible.

We demonstrate the pattern in a demanding testbed: real-time management of a physics-informed Kolmogorov-Arnold network (PI-KAN) solving the Heston stochastic volatility PDE, where the C^2 continuity requirement of the cross-derivative term creates a hard mathematical invariant the agent must not violate. A dual-process architecture — 7B reflexive path (<50 ms) and 32B strategic path (200–500 ms) — operates concurrently with the GPU training loop with zero blocking.

Evaluation against five financial monotonicity checks reveals that PDE residual training alone is *insufficient* for regime-ordered behavior — monotonicity requires the semantic controller — identifying training objective insufficiency as a general failure mode in physics-constrained networks under distributional shift. Protocol benchmarks achieve 1.00 execution rate and 0.75 semantic agreement against human-authored gold DSL, with the residual gap identifying a normalization contract problem rather than a comprehension failure. Sustained throughput reaches **26,300** smp/sec on NVIDIA H200 with zero GPU blocking.

Contents

1 Introduction **4**
1.1 The Interface Problem: LLM-Driven Topology Management 4
1.2 The Semantic Compiler Approach 4
1.3 The Testbed: Why Heston/KAN Is a Demanding Domain 4
1.4 Contributions 5
1.5 Scope and Positioning 5
1.6 Paper Organization 5

2 Related Work **5**
2.1 Physics-Informed Learning for Financial PDEs 6
2.2 Neural Architecture Search and Topology Management 6
2.3 Kolmogorov-Arnold Networks 6

2.4 LLM Tool Use and Financial NLP . . . 6
2.5 Safe and Sandboxed Code Execution . . . 7
2.6 Market Regime Detection . . . 7
2.7 Dual-Process AI Architectures . . . 7

3 Problem Formulation **7**
3.1 The Heston PDE and the C^2 Constraint . . . 7
3.2 The Regime Problem: Why a Single Learned Model Is Insufficient . . . 8
3.3 The Narrative Problem: Why Structured Models Cannot Self-Update . . . 8

4 The Semantic Compiler Pattern **8**
4.1 The Core Problem: Type Safety at the LLM-Network Boundary . . . 9
4.2 Pattern Definition . . . 9
4.3 Why This Over the Alternatives . . . 9
4.4 Generalizability . . . 10

5 Architecture **10**
5.1 Design Philosophy: One Problem per Layer . . . 10
5.2 Layer 1: The Physics-Informed KAN . . . 10
5.2.1 Why KAN Over MLP for Heston . . . 10
5.2.2 B-Spline Edge Activations . . . 11
5.2.3 Model Architecture and Training Objective . . . 12
5.3 Layer 2: Hidden Markov Regime Detector . . . 12
5.3.1 Why a Separate Regime Layer . . . 12
5.3.2 Features and Walk-Forward Inference . . . 12
5.4 Layer 3: The Topology Management Agent . . . 12
5.4.1 The Core Idea: LLM as Semantic Compiler . . . 12
5.4.2 Two-Speed Architecture . . . 13
5.4.3 The Domain-Specific Language . . . 13
5.5 End-to-End Coordination . . . 14

6 Correctness Guarantees **14**
6.1 The Code Injection Problem . . . 14
6.2 AST Allowlist Validation . . . 14
6.3 Correct Pruning Semantics . . . 15
6.4 Atomic Mutation Rollback . . . 16

7 Experimental Investigation **16**
7.1 Research Questions . . . 16
7.2 Experimental Setup . . . 17
7.3 Protocol Benchmark Tiers . . . 17

8 Results and Findings **18**
8.1 RQ1 – Protocol Validity . . . 18
8.2 RQ2 – Paraphrase Stability . . . 18
8.3 RQ3 – Normalization Gap . . . 19
8.4 RQ4 – Regime Coherence . . . 19
8.5 RQ5 – Pricing Performance . . . 20
8.6 RQ6 – Engineering Performance . . . 21

9 Discussion **21**
9.1 What the Architecture Gets Right . . . 21
9.2 The Monotonicity Gap: A Training Objective Problem . . . 22

9.3 The Normalization Gap: A Contract Problem 22
9.4 Cross-Asset Transfer: What Domain Specificity Means 22

10 Limitations and Future Directions **22**

11 Conclusion **23**

1 Introduction

1.1 The Interface Problem: LLM-Driven Topology Management

Consider a neural network solving a physics PDE that is currently training on a GPU. An analyst reads a market narrative and wants the network's topology updated to reflect the new regime — specific edges replaced with symbolic functions, others pruned — within seconds, without stopping training. This is a topology management problem, and giving an LLM control over it creates a specific interface challenge that has no satisfactory solution in existing literature.

The mismatch is structural. The LLM produces natural language or loosely-structured JSON; the network accepts only typed, mathematically valid structural mutations. Three requirements must be simultaneously satisfied: (1) **Type safety**: the mutation must conform to a well-defined schema before it reaches the network; (2) **Vocabulary restriction**: any formula proposed must be limited to a safe mathematical vocabulary that preserves the network's invariants (in our case, C^2 continuity for Heston PDE evaluation); (3) **Transactional atomicity**: if a mutation batch fails mid-execution, the model must be restored to its pre-mutation state.

Naive approaches fail all three simultaneously. Direct evaluation of LLM-generated Python via `eval()` violates type safety (no schema enforcement), vocabulary restriction (any Python builtin is accessible), and atomicity (partial mutations persist on failure). Neural architecture search [Elsken et al., 2019] solves none of these problems: it operates offline before training, receives no natural language input, and provides no mechanism for enforcing domain-specific mathematical invariants. Prompt-to-price approaches — where the LLM produces a numerical price directly — are not topology management at all: they bypass the learned model entirely and introduce hallucination risk on numerical outputs.

1.2 The Semantic Compiler Approach

The semantic compiler resolves this by positioning the LLM as a compiler: it takes unstructured natural language and produces structured, typed output. The "compilation" step enforces three properties before the output ever reaches the network:

- **Pydantic schema validation at parse time.** The LLM is instructed to produce JSON conforming to a typed schema; any output that fails validation triggers an error-correction retry rather than downstream execution.
- **AST allowlist analysis on formula strings.** Any formula string within a command is checked by static analysis against a pre-approved mathematical vocabulary before it is stored. The vocabulary is domain-specific: it encodes the set of functions that are C^2 continuous on their natural domains, blocking all other names including module imports and file-system access.
- **deepcopy snapshot + rollback for transactional integrity.** All affected modules are snapshotted before mutation begins. Any exception during application restores all objects. Either all mutations in a batch succeed or none do.

The LLM never produces a price, never modifies weights directly, and never executes code. It produces a DSL object. The specialist backend handles execution.

1.3 The Testbed: Why Heston/KAN Is a Demanding Domain

The finance domain is not the primary contribution of this paper, but it is chosen as a testbed because it stress-tests the semantic compiler pattern in three ways that synthetic benchmarks cannot:

- **C^2 continuity constraint.** The agent must propose formulas that are twice-differentiable

on the physical domain, not merely syntactically valid Python. This makes the vocabulary layer's mathematical content non-trivial: the allowlist encodes a genuine mathematical property, not just a security policy.

- **PDE geometry reasoning.** Strategic topology decisions require understanding which edge activations are consistent with the Heston differential operator's structure — a reasoning task that exercises the 32B strategic model's domain comprehension at each decision step.
- **Real-time latency.** A System-2 decision cannot block GPU training, motivating the dual-process architecture. The asynchronous design is not a performance optimization; it is a correctness requirement for any online topology management system.

1.4 Contributions

This paper makes the following contributions:

1. **The semantic compiler design pattern** (Typed DSL + AST allowlist + atomic rollback) as a reusable approach for safe LLM-driven topology management of physics-constrained networks.
2. **A dual-process non-blocking architecture** (System 1 reflexive, System 2 strategic) with a formal zero-blocking guarantee enforced by bounded queues and non-blocking drain logic.
3. **PI-KAN for Heston**: a complete implementation with C^2 B-splines via Cox–de Boor recursion, verified by second-order autograd on the Heston cross-derivative term.
4. **A tiered evaluation framework** spanning protocol validity, paraphrase stability, normalization gap, regime coherence, pricing performance, and engineering headroom — designed to independently assess each layer of the architecture.
5. **The training objective insufficiency finding**: PDE residual minimization alone does not produce regime-ordered behavior. Monotonicity requires the semantic controller, identifying a general failure mode in physics-constrained networks under distributional shift.

1.5 Scope and Positioning

OpKAN is not a pricing formula and does not claim to outperform all existing pricing models in all conditions. Its specific claim is that the semantic compiler pattern makes LLM-driven topology management safe, auditable, and non-blocking in a domain where violations of mathematical invariants have immediate operational consequences. The cross-asset experiments in Section 8 show precisely where the architecture's advantages hold and where domain coverage limits them.

1.6 Paper Organization

Section 2 situates the work in the literature. Section 3 develops the problem formulation. Section 4 defines the semantic compiler pattern in the abstract. Section 5 describes the OpKAN instantiation. Section 6 presents correctness guarantees. Section 7 presents the experimental design. Section 8 reports findings. Section 9 interprets them. Section 10 states scope boundaries and outlines future directions. Section 11 concludes.

2 Related Work

OpKAN sits at the intersection of six research areas. We review each with emphasis on the specific gaps that motivated our design choices.

2.1 Physics-Informed Learning for Financial PDEs

PINNs [Raissi et al., 2019] convert a PDE boundary value problem into an unconstrained optimization by embedding the differential operator residual in the loss function. Applications to financial PDEs [Horvath et al., 2021] have generally used standard MLP backbones. The C^2 continuity requirement of the Heston cross-derivative term has received limited attention; in practice, many implementations approximate or omit the $\rho\sigma vS$ coefficient entirely. OpKAN's PI-KAN backbone addresses this gap by using activations whose continuity class is guaranteed by construction.

Finite difference methods [Wilmott et al., 1993] and Monte Carlo simulation remain standard Heston baselines. They are accurate but rigid: re-solving under shifted parameters requires re-running the full solver. Deep hedging [Bühler et al., 2019] extends the neural approach to replication and hedging under real-world dynamics; Neural SDEs [Kidger et al., 2021] provide continuous-time neural network models that are related architecturally to our PDE-constrained training but do not enforce specific boundary conditions in the Heston sense.

2.2 Neural Architecture Search and Topology Management

Neural architecture search (NAS) [Elsken et al., 2019] automates the discovery of optimal network topologies through gradient-based [Liu et al., 2019] or evolutionary search. The semantic compiler approach is architecturally distinct along three axes. First, NAS operates *offline*: it finds a fixed architecture before training begins, whereas the topology agent performs *online* surgery on a network that is actively training. Second, NAS receives no natural language input; the architecture space is explored algorithmically with no semantic grounding. Third, NAS does not enforce domain-specific mathematical invariants during search: there is no mechanism to require that discovered activations be C^2 continuous, or that a proposed edge replacement be consistent with PDE geometry.

Continual and lifelong learning [Kirkpatrick et al., 2017] addresses a related problem — how to modify a network without catastrophic forgetting — but does not address the interface between natural language and topology mutation. The semantic compiler does not manage forgetting; it manages the safety and correctness of the mutation itself.

2.3 Kolmogorov-Arnold Networks

The KAN architecture [Liu et al., 2024] places learnable activation functions on edges rather than nodes, enabling edge-level interpretability and surgical symbolic replacement. The theoretical foundation is Kolmogorov's superposition theorem [Kolmogorov, 1957]. The key practical advantage over MLPs is the combination of C^2 B-spline activations and the ability to replace individual edges with symbolic functions.

MonoKAN [Polo-Molina et al., 2024] extends KANs with certified monotonicity constraints — the direction OpKAN's architecture should evolve. The current system achieves regime-ordered pricing through an external semantic controller, not native architectural monotonicity; selective MonoKAN substitution is a stated future direction (Section 10).

2.4 LLM Tool Use and Financial NLP

Toolformer [Schick et al., 2023] and ReAct [Yao et al., 2023] establish that LLMs can learn to invoke external tools and APIs. Domain-specific financial LLMs such as FinBERT [Huang et al., 2023] and BloombergGPT [Wu et al., 2023] demonstrate that pre-training on financial text improves structured information extraction.

OpKAN's topology management agent takes a different position from all of these: the LLM is a *source of structured commands*, not a direct decision-maker or a named-entity extractor. Its output is a typed DSL object validated by a Pydantic schema before it is applied, and the validation failure itself is auditable. This distinction has operational consequences: a generic tool call can fail silently or produce financially inconsistent outputs; a DSL command cannot be applied if it fails schema validation.

2.5 Safe and Sandboxed Code Execution

The problem of safely executing LLM-generated code has been approached through runtime sandboxes (restricted interpreter environments, OS-level isolation) and static analysis (type checking, taint tracking). Runtime sandboxes impose significant overhead and cannot enforce domain-specific constraints such as C^2 continuity. Static type checking operates at a coarser granularity than expression-level allowlisting.

The AST allowlist approach in Section 6 occupies a different point in this design space: it is a *compile-time domain grammar restriction* that doubles as a mathematical vocabulary constraint. Rather than asking "is this code safe to execute?" (the sandbox question), it asks "is this expression within the mathematical vocabulary I have pre-approved for C^2 network activations?" The vocabulary constraint is tighter than any general-purpose sandbox while being lighter-weight and directly enforceable at parse time.

2.6 Market Regime Detection

Gaussian HMMs for market regime detection follow Hamilton's foundational work [Hamilton, 1989] on nonlinear regime-switching time series. The label stability problem under rolling re-estimation is well known but rarely addressed explicitly in applied work. Our realized-volatility sorting approach (Section 5.3) is grounded in the empirical regularity that realized volatility is the most discriminative feature for separating market regimes in our feature set.

2.7 Dual-Process AI Architectures

Kahneman's dual-process framework [Kahneman, 2011] has been applied to AI agent design in robotics and game-playing. The specific combination of an LLM fast path and an LLM slow path operating on the same bounded queues — with both paths producing typed, schema-validated outputs that feed the same mutation pipeline — is, to our knowledge, a novel application to neural network topology management.

3 Problem Formulation

Before describing the architecture, we formalize the constraints that any solution must satisfy. This section establishes three requirements that jointly motivate the design choices in Section 5.

3.1 The Heston PDE and the C^2 Constraint

The Heston stochastic volatility model [Heston, 1993] describes risk-neutral asset dynamics as a joint process for spot price S_t and instantaneous variance v_t:

$$dS_t = rS_t\,dt + \sqrt{v_t}S_t\,dW_t^S \tag{1}$$

$$dv_t = \kappa(\theta - v_t)\,dt + \sigma\sqrt{v_t}\,dW_t^v \tag{2}$$

where κ is mean-reversion speed, θ is the long-run variance, σ is volatility-of-volatility, and $\mathrm{Corr}(dW^S, dW^v) = \rho\,dt$ captures the leverage effect. By Itô's lemma, the fair value $V(S, v, t)$ of

a European call satisfies the Heston PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}vS^2\frac{\partial^2 V}{\partial S^2} + \rho\sigma vS\frac{\partial^2 V}{\partial S\partial v} + \frac{1}{2}\sigma^2 v\frac{\partial^2 V}{\partial v^2} + rS\frac{\partial V}{\partial S} + \kappa(\theta - v)\frac{\partial V}{\partial v} - rV = 0 \tag{3}$$

supplemented by boundary conditions:

$$V(S, v, T) = \max(S - K, 0) \quad \text{(terminal payoff)} \tag{4}$$

$$V(0, v, t) = 0 \quad \text{(zero spot)} \tag{5}$$

$$V(S, v, t) \approx S - Ke^{-r(T-t)} \quad \text{as } S \to \infty \tag{6}$$

The cross-derivative term $\rho\sigma vS\,\partial^2 V/\partial S\partial v$ is the key constraint for architecture design. Evaluating this term via automatic differentiation requires the network activation functions to be *at least C^2 continuous*: once-differentiable activations (ReLU, leaky ReLU) produce zero second derivatives and make this term numerically degenerate, ruling out standard MLP architectures for faithful Heston PINN training.

3.2 The Regime Problem: Why a Single Learned Model Is Insufficient

A single trained neural network, even a well-constrained PINN, learns a *fixed* mapping from inputs (S, v, t) to prices. The Heston parameters $(\kappa, \theta, \sigma, \rho)$ are not constants — they shift with market regime. A single frozen model trained on historical data averaged across regimes will systematically underprice out-of-the-money options in stress conditions and overprice them in calm conditions. The regime problem is a consequence of the non-stationarity of the underlying stochastic process, not a defect of any particular architecture.

3.3 The Narrative Problem: Why Structured Models Cannot Self-Update

Even with a regime-sensitive architecture, a purely quantitative system has no channel for the most time-sensitive market information: qualitative narrative. Central bank announcements, geopolitical developments, and supply-shock narratives arrive as natural language. Converting them to structured inputs requires either a human analyst (high latency, not scalable) or a system that reliably translates unstructured text into structured quantitative commands.

This is not primarily a machine learning problem — it is an interface design problem. The challenge is to define an interface that accepts free-form language as input and produces structured, validated quantitative commands as output, so that the downstream pricing model never receives unvalidated, potentially inconsistent inputs.

These three requirements — C^2 continuous activations, regime-adaptive topology, and a structured interface for narrative inputs — jointly motivate the semantic compiler pattern in Section 4 and its OpKAN instantiation in Section 5.

4 The Semantic Compiler Pattern

Before describing the OpKAN instantiation, we abstract the design pattern it implements. The pattern applies to any system where an LLM must modify the structure of a running constraint-satisfying computational process. The Heston PI-KAN is one instance; the pattern is reusable wherever similar interface requirements arise.

4.1 The Core Problem: Type Safety at the LLM-Network Boundary

The fundamental mismatch is representational. An LLM's natural output space is unstructured text or loosely-structured JSON; a physics-constrained neural network's mutation interface requires typed, validated commands. Three failure modes occur when this boundary is not enforced:

1. **Schema failure.** The LLM produces a structurally invalid command (missing required field, wrong type). Without schema validation, this reaches the mutation pipeline and causes a runtime error at an unpredictable depth.
2. **Vocabulary failure.** The LLM proposes a formula string containing operations outside the safe mathematical vocabulary (e.g., importing a module, accessing the file system, calling a disallowed function). Without static analysis, this executes with the full privileges of the training process.
3. **Atomicity failure.** A mutation batch applies successfully to the first three edges and fails on the fourth. Without rollback, the model is left in a partially-mutated state with no record of which edges were modified.

The semantic compiler resolves all three.

4.2 Pattern Definition

Definition 1 (Semantic Compiler)**.** *A* semantic compiler *is an LLM-to-backend interface with three enforcement layers:*

1. ***Type layer.*** *A Pydantic schema defines the grammar of valid commands. The LLM is instructed to produce JSON conforming to this schema; any output that fails schema validation triggers an error-correction retry rather than downstream execution.*
2. ***Vocabulary layer.*** *For any formula string within a command, a static AST analysis pass verifies membership in a pre-approved vocabulary before the formula is stored. The vocabulary is domain-specific: in OpKAN it encodes the set of functions that are C^2 continuous on their natural domains.*
3. ***Transaction layer.*** *All affected backend objects are snapshotted via* `deepcopy` *before mutation begins. Any exception during application restores all objects and logs the failure. Either all mutations in a batch succeed or none do.*

Together, these three layers provide a formal guarantee: any command that reaches the backend is *well-typed* (passes Pydantic), *vocabulary-safe* (passes AST analysis), and *applied atomically* (transactional rollback). The LLM never produces a price, never modifies weights directly, and never executes code; it produces a DSL object. All execution happens in a typed, validated, reversible backend.

4.3 Why This Over the Alternatives

Direct LLM output (price or weight update). The LLM produces a numerical price or a weight delta directly. This removes the interface entirely but eliminates auditability, introduces hallucination risk on numerical outputs, and cannot represent structured topology operations such as edge pruning or symbolic replacement.

Prompt-to-eval() (LLM generates Python, eval runs it). The LLM generates arbitrary Python that is evaluated at runtime. This is a code injection vulnerability. A prompt-injected formula string `__import__("os").system("rm -rf /")` executes with the full privileges of the training process. Runtime sandboxes add overhead without enforcing domain-specific mathematical vocabulary.

Neural architecture search. NAS discovers optimal topology offline through gradient or evolutionary search. It does not accept natural language as input, does not operate on a running network, and does not enforce domain constraints during search. NAS and the semantic compiler are complementary, not alternatives: NAS could be used to initialize the topology; the semantic compiler manages it at runtime.

4.4 Generalizability

The semantic compiler pattern is not finance-specific. Any domain where an LLM must modify a running constraint-satisfying system — a chemical synthesis simulator where reaction pathways must satisfy thermodynamic constraints, a physical simulation where mesh refinements must preserve conservation laws, a robotics controller where trajectory modifications must remain within kinematic limits — presents the same three-layer interface problem. The OpKAN instantiation demonstrates viability in a domain where the constraints are mathematically sharp and the failure modes are operationally consequential.

5 Architecture

5.1 Design Philosophy: One Problem per Layer

The OpKAN architecture is organized around a single principle: each layer should do exactly one thing well, and the boundaries between layers should be explicit, typed, and enforceable.

- **Layer 1 (PI-KAN):** Solve the Heston PDE. Know nothing about market regimes or LLM outputs.
- **Layer 2 (HMM):** Produce a stable regime label from market microstructure features. Know nothing about network topology or language.
- **Layer 3 (Topology Agent):** Translate regime signals and topology statistics into typed mutation commands. Know nothing about the PDE internals or the raw market data.

Figure 1 shows the information flow. Data moves in one direction (market data → features → regime label → updated network), with the agent operating asynchronously so it never blocks the GPU training loop.

5.2 Layer 1: The Physics-Informed KAN

5.2.1 Why KAN Over MLP for Heston

The Kolmogorov-Arnold representation theorem [Kolmogorov, 1957] guarantees that any continuous function $f : [0,1]^n \to \mathbb{R}$ can be expressed as a composition of univariate continuous functions:

$$f(x_1, \ldots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right) \tag{7}$$

The domain in Eq. (7) is the unit hypercube $[0,1]^n$; in practice the training domain $(S, v, t) \in [0, 200] \times [0, 0.5] \times [0, 1]$ is bounded and the collocation grid is normalized accordingly. The KAN architecture [Liu et al., 2024] operationalizes this by placing learnable activation functions on *edges* rather than nodes. This has two concrete consequences for our problem.

First, edge activations can be parameterized as B-splines, which are *genuinely C^2 continuous*, satisfying the Heston cross-derivative requirement that standard MLPs cannot meet.

Second, individual edges can be replaced with closed-form symbolic expressions without modifying the rest of the network — the topological handle the agent uses to inject domain knowledge.

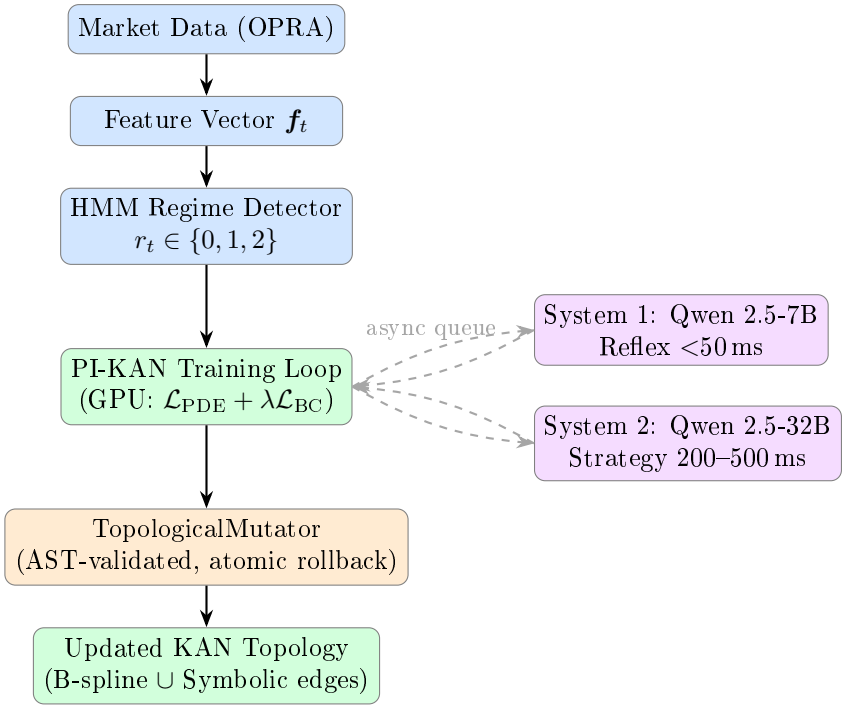

Figure 1: OpKAN information flow across three decoupled layers. The HMM does not know about the KAN; the topology agent does not know about the PDE; the GPU training loop never waits on the LLM. Dashed arrows are asynchronous (bounded queues; maxsize=1 for inputs, maxsize=32 for outputs). The full training throughput of 26,300 smp/sec is measured during sustained 5-epoch training; the standalone E2E validation suite reports 6,506 smp/sec under a 50-step cold-start microbenchmark (see Section 8.6).

The resulting network, where some edges are B-splines and some are closed-form functions, is referred to as a **Symbolic PI-KAN**.

5.2.2 B-Spline Edge Activations

Each edge is implemented as a `BSplineEdge` module. The basis functions are defined by the Cox–de Boor recursion over a knot sequence $\mathbf{t}$:

$$B_{i,0}(x) = \mathbf{1}[t_i \le x < t_{i+1}] \tag{8}$$

$$B_{i,k}(x) = \frac{x - t_i}{t_{i+k} - t_i} B_{i,k-1}(x) + \frac{t_{i+k+1} - x}{t_{i+k+1} - t_{i+1}} B_{i+1,k-1}(x) \tag{9}$$

OpKAN uses $G = 5$ interior intervals and spline order $k = 3$ (cubic), giving $G + k = 8$ basis functions per edge. The edge activation combines a SiLU base with a trained spline correction:

$$\phi(x) = w_{\text{base}} \cdot \text{SiLU}(x) + w_{\text{spline}} \cdot \sum_{i=1}^{8} c_i B_{i,3}(x) \tag{10}$$

where $w_{\text{base}}, w_{\text{spline}} \in \mathbb{R}$ and $\{c_i\}_{i=1}^{8}$ are trainable. Because each $B_{i,3}$ is C^2 by construction, Eq. (10) is C^2 for any choice of coefficients.

An edge with near-zero spline coefficients can be *pruned* (replaced by a `ZeroEdge` that returns `torch.zeros_like(x)`); an edge whose activation profile approximates a smooth function can be *replaced* by a `C2SymbolicEdge`.

5.2.3 Model Architecture and Training Objective

The PI-KAN uses a $[3, 16, 1]$ configuration (three inputs (S, v, t), 16-node hidden layer, scalar output V). The forward pass aggregates edge outputs by summation:

$$h_j^{(\ell)} = \sum_{i=1}^{n_{\text{in}}^{(\ell)}} \phi_{ij}^{(\ell)} \left(h_i^{(\ell-1)} \right) \tag{11}$$

Training minimizes a composite physics-informed loss over randomly sampled collocation points:

$$\mathcal{L} = \underbrace{\mathbb{E}_{\boldsymbol{x}\sim\Omega}\left[|\mathcal{N}[V](\boldsymbol{x})|^2 \right]}_{\mathcal{L}_{\text{PDE}}} + \lambda_{\text{BC}} \underbrace{\mathbb{E}_{\boldsymbol{x}\sim\partial\Omega}\left[|V(\boldsymbol{x}) - V_{\text{target}}(\boldsymbol{x})|^2 \right]}_{\mathcal{L}_{\text{BC}}} \tag{12}$$

where $\mathcal{N}[V]$ is the Heston differential operator evaluated via `torch.autograd` with `create_graph=True` for second-order computation. Interior collocation points $\boldsymbol{x} = (S, v, t)$ are drawn uniformly from $[0, 200] \times [0, 0.5] \times [0, 1]$ each batch. A critical implementation note: these points are sampled randomly and *independently of observed market data*. Using market-observed (S, v, t) triples as collocation points would concentrate the PDE residual in historically observed regions, biasing the model away from the full domain. This look-ahead bias was present in earlier versions of this codebase and is corrected here; a quantitative before/after ablation is deferred to future work.

5.3 Layer 2: Hidden Markov Regime Detector

5.3.1 Why a Separate Regime Layer

Why not let the PI-KAN learn regime-sensitivity directly from data? The Heston PDE does not change with regime — it is a structural prior on the *shape* of the pricing function. What changes with regime is the *parameters* κ, σ, ρ. Conflating these two levels of adaptation would force the PI-KAN to simultaneously learn a PDE solution and a parameter estimation procedure, undermining the physics prior. A separate regime detector provides a clean conditioning signal.

5.3.2 Features and Walk-Forward Inference

The regime detector fits a Gaussian HMM with diagonal covariance to five time-series features:

- $\ell_t = \log(S_t/S_{t-1})$ (log-return)
- $\hat{\sigma}_t = \text{std}(\ell_{t-W:t}) \cdot \sqrt{252 \times 6.5 \times 60}$ (realized vol, $W = 20$)
- σ_t^{IV} (implied volatility from the options chain)
- $\Delta_t = \sigma_t^{\text{IV}} - \hat{\sigma}_t$ (IV–RV spread)
- $\mu_t^{\text{IV}} = \sigma_t^{\text{IV}} - \sigma_{t-1}^{\text{IV}}$ (IV momentum)

A persistent problem with rolling HMM inference is state label instability: "state 0" in window t may map to "state 1" in window $t+1$ after a refit. We fix this with a realized-volatility sorting pass:

$$\pi = \text{argsort}(\boldsymbol{\mu}_{:,1}), \quad \text{remap}[s] = \pi^{-1}[s] \tag{13}$$

where $\boldsymbol{\mu}_{:,1}$ is the vector of per-state mean realized volatilities, ensuring that label 0 always maps to the lowest-volatility regime across all rolling windows.

5.4 Layer 3: The Topology Management Agent

5.4.1 The Core Idea: LLM as Semantic Compiler

The topology management agent, internally named the *LiuClaw* agent, addresses a fundamental interface problem. On one side is unstructured text; on the other is a quantitative backend

that accepts only typed mutation commands (prune this edge, replace that activation with this formula). The agent's job is to translate one representation into the other — making it, in the precise sense of the term, a *semantic compiler*.

The crucial design constraint is type-safety at parse time. The LLM is instructed to generate JSON conforming to the `StrategicDecision` or `ReflexDecision` Pydantic schema; the schema validates the output before it enters the mutation pipeline. A malformed response triggers an error-correction retry rather than a downstream failure.

5.4.2 Two-Speed Architecture

The agent operates on two timescales, motivated by the qualitatively different tasks each handles.

Fast path (System 1 — Reflexive Maintenance). Every 10 training steps, the training loop places the step index, per-edge L1 activation norms, and the recent loss delta into a bounded queue. A background thread dequeues this, passes it to a 7B-parameter model (Qwen 2.5-7B), and writes back a `ReflexDecision`:

```
class ReflexDecision(BaseModel):
    reasoning: str        # brief justification
    prunes: List[str]     # edge IDs with near-zero L1 norm
    lr_adjustment: float  # learning-rate multiplier in [0.5, 2.0]
```

This path prunes low-norm edges and adjusts the learning rate. Latency must stay below 50 ms, which is why the smallest available model is used.

Slow path (System 2 — Strategic Surgery). Every 100 training steps, a longer-horizon context — loss trajectory, HMM transition probabilities, full KAN state — is placed in a separate bounded queue. A second background thread passes it to a 32B-parameter model (Qwen 2.5-32B) and writes back a `StrategicDecision`:

```
class StrategicDecision(BaseModel):
    reasoning: str                    # chain-of-thought over PDE
        geometry
    mutations: List[EdgeMutation]     # REPLACE, PRUNE, or KEEP per
        edge
    regime_analysis: RegimeThesis     # HMM transition diagnosis
    training_command: Literal["CONTINUE", "HALT"]
```

This path replaces B-spline edges with symbolic functions that the agent reasons are consistent with the Heston PDE geometry. The 32B model is used because edge replacement requires coherent mathematical reasoning across multiple PDE constraints.

This bifurcation implements Kahneman's dual-process model [Kahneman, 2011]: System 1 (fast, cheap, pattern recognition) never blocks System 2 (slow, deliberate, multi-step reasoning).

5.4.3 The Domain-Specific Language

Every agent output is a JSON object conforming to a Pydantic schema. The atomic unit is the `EdgeMutation`:

- `edge_id`: form `L{layer}_N{in}_to_L{layer+1}_N{out}`
- `action`: one of `PRUNE`, `REPLACE`, `KEEP`
- `formula`: for `REPLACE`, a PyTorch expression from the allowlisted vocabulary
- `initial_params`: optional `{scale, shift, coeff}` for warm initialization
- `reasoning`: quantitative justification citing L1 norms and sampled activation values

To make the interface concrete, the following is a representative end-to-end trace: an analyst submits the text *"COMEX silver inventories fell 8% overnight; demand shock likely"*, and the agent emits:

```
{
  "training_command": "CONTINUE",
  "regime_analysis": {"hmm_transition_detected": true,
                      "predicted_regime": 2,
                      "thesis_statement": "Inventory␣shock..."},
  "mutations": [
    {"edge_id": "L0_N0_to_L1_N3",
     "action": "REPLACE",
     "formula": "torch.exp(-torch.abs(x))",
     "reasoning": "L1␣norm␣0.0031␣<␣0.005;␣shape␣matches␣decaying␣exponential",
     "initial_params": {"scale": 1.0}}
  ]
}
```

The schema validates this JSON object (types, Literal constraints, field presence) before it reaches the mutation pipeline; it is rejected and retried if any field violates the schema.

5.5 End-to-End Coordination

The `EngineCoordinator` manages four bounded queues:

- `reflex_queue` (maxsize=1): training loop → System 1 thread. Latest-context-only: old context discarded if not yet consumed.
- `reflex_decision_queue` (maxsize=32): System 1 thread → training loop.
- `strategic_queue` (maxsize=1): training loop → System 2 thread.
- `strategic_decision_queue` (maxsize=32): System 2 thread → training loop.

At the start of each training batch, `apply_pending_mutations(model, optimizer)` non-blockingly drains both output queues. If no decisions are pending, training continues at full GPU throughput. The GPU never waits on the LLM.

6 Correctness Guarantees

The topology agent introduces an attack surface that conventional neural networks do not have: it evaluates LLM-generated Python expressions inside a running training process.

6.1 The Code Injection Problem

The original implementation evaluated formula strings via Python's `eval()`:

```
out = eval(self.expression_str, {"torch": torch}, {"x": x})
```

This is trivially exploitable. A formula string `'...__import__("os").system("rm -rf /")'` executes a shell command at evaluation time with the full privileges of the training process. Since the formula string originates from an LLM — which could be adversarially prompted or subject to prompt injection from document inputs — this is a critical vulnerability in any production deployment.

6.2 AST Allowlist Validation

We fix this with a static AST analysis pass that runs *before the expression is stored*:

```python
_ALLOWED_TORCH_FUNCS = frozenset({
    'sin', 'cos', 'tan', 'exp', 'log', 'log1p', 'sqrt', 'abs',
    'pow', 'tanh', 'sigmoid', 'softplus', 'sign', 'clamp',
    'relu', 'floor', 'ceil', 'round', 'reciprocal', 'neg'
})
_SAFE_AST_NODES = frozenset({
    ast.Expression, ast.Call, ast.BinOp, ast.UnaryOp,
    ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Pow,
    ast.FloorDiv, ast.Mod, ast.USub, ast.UAdd,
    ast.Constant, ast.Load
})

def _validate_symbolic_expression(expr: str) -> None:
    tree = ast.parse(expr, mode='eval')
    for node in ast.walk(tree):
        if type(node) in _SAFE_AST_NODES: continue
        if isinstance(node, ast.Name):
            if node.id not in ('x', 'torch'):
                raise ValueError(f"Disallowed␣name␣'{node.id}'")
        elif isinstance(node, ast.Attribute):
            if not (isinstance(node.value, ast.Name)
                    and node.value.id == 'torch'):
                raise ValueError("Only␣'torch.*'␣attributes␣allowed.")
            if node.attr not in _ALLOWED_TORCH_FUNCS:
                raise ValueError(f"torch.{node.attr}␣not␣in␣allowlist."
                    )
        else:
            raise ValueError(f"Disallowed␣node␣'{type(node).__name__}'"
                )
```

Proposition 1. *Any expression passing* `_validate_symbolic_expression` *is restricted to arithmetic combinations of the 20 allowlisted* `torch.*` *functions applied to the variable* x*. It cannot import modules, access the file system, or invoke arbitrary Python builtins.*

Proof sketch. The `ast.walk` traversal visits every node in the parse tree. The validator partitions all possible AST node types into three exhaustive cases and raises on any unexpected case: *(i)* safe structural nodes (`_SAFE_AST_NODES`) are passed through; *(ii)* `ast.Name` nodes are permitted only for identifiers `x` and `torch`, blocking all other variable references and built-in names (including `__import__`, `eval`, `open`); *(iii)* `ast.Attribute` nodes are permitted only as `torch.{attr}` where `attr` is in the 20-function allowlist, blocking chained attribute access (`__class__.__bases__`). Any node type not in these three cases raises immediately. No path to module import, file I/O, or builtins survives the traversal. □

Remark 1. *The allowlist vocabulary was designed around C^2 continuity requirements: all 20 functions are smooth on their natural domains. A future hardened version should additionally verify differentiability at runtime by probing second derivatives on a sample grid.*

6.3 Correct Pruning Semantics

An earlier implementation represented a pruned edge as `nn.Sequential()`, the identity under default PyTorch, making `PRUNE` semantically equivalent to `KEEP`. We replace this with:

```python
class ZeroEdge(nn.Module):
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.zeros_like(x)
```

`ZeroEdge` correctly removes the edge's contribution from downstream aggregation, enabling genuine network sparsification.

6.4 Atomic Mutation Rollback

A mutation batch may fail mid-execution if a formula passes AST validation but fails at runtime. Without rollback, the model is left in a *partially mutated* state. We prevent this by snapshotting all affected edge modules via `copy.deepcopy` before applying any mutations and restoring them on any exception:

Algorithm 1 Atomic Mutation Application with Rollback

```
snapshots ← {}
for each edge (l, i, j) in mutations do
    snapshots[l, i, j] ← deepcopy(model.layers[l].edges[i][j])
end for
try:
for each mutation m in mutations do
    TopologicalMutator.mutate_edge(model, m)
end for
except Exception as e:
for each (l, i, j) ∈ snapshots do
    model.layers[l].edges[i][j] ← snapshots[l, i, j]
end for
log rollback event with exception e
```

`state_dict` snapshots are insufficient because `swap_edge` replaces entire `nn.Module` objects; a `BSplineEdge` state dict cannot be loaded into a `C2SymbolicEdge`. We snapshot module objects rather than parameter tensors.

7 Experimental Investigation

7.1 Research Questions

Our experiments are preliminary investigations into six questions targeting specific layers of the architecture. We frame them as questions rather than hypotheses to signal that the current results raise follow-up problems rather than resolving them.

RQ1 Protocol validity. Can the agent reliably compile direct and narrative prompts into valid, executable DSL while preserving the analyst's domain intent? Does performance degrade gracefully or catastrophically with longer document context?

RQ2 Paraphrase stability. Is the compiled DSL stable under paraphrase? Does rewording the same analyst intent produce a functionally equivalent DSL command?

RQ3 Normalization gap. How large is the gap between the agent producing a *valid* DSL command and producing the *same* DSL command a human analyst would have produced?

RQ4 Regime coherence. Does routing a fixed query through STABLE, TRANSITION, and STRESS regime contexts produce directionally ordered backend outputs? This tests whether the explicit semantic controller works correctly and whether PDE residual minimization alone is sufficient for regime ordering.

RQ5 Pricing performance. Which combination of physics model and learned residual produces the lowest price MAE in each regime? Is the hybrid advantage consistent across

assets?

RQ6 Engineering headroom. Does the asynchronous dual-queue architecture maintain full GPU training throughput under concurrent LLM interaction?

7.2 Experimental Setup

Hardware. All GPU experiments run on NVIDIA H200 with CUDA. LLM inference uses Ollama (local) with `qwen2.5-coder` for protocol benchmarks and vLLM for the engineering stress test. We use a code-specialized model for the protocol benchmarks because the DSL has strongly code-like syntax (typed JSON schema, identifier conventions), and code models demonstrate superior structured-output adherence in our preliminary tests.

Data. The pricing benchmarks use SLV (silver ETF) option chains with real historical regime features. The live benchmark uses one quote date (cross-sectional split). The replay benchmark uses a historical silver/regime feature sequence with a replayed SLV surface template, enabling evaluation across stable, transition, and stress dates.

Model families. We compare six families. The `spikan_*` variants all use a Symbolic PI-KAN (B-spline edges with some agent-replaced symbolic edges) as the base model; they differ in their pricing formula anchor and residual type:

- `bs_plain`: Black-Scholes with baseline ATM volatility.
- `bs_regime`: Black-Scholes with HMM regime-predicted volatility.
- `rf_direct`: Random forest direct price prediction from structured features.
- `spikan_hybrid`: Symbolic PI-KAN residual added to a *regime-vol* Black-Scholes anchor.
- `spikan_plain_hybrid`: Symbolic PI-KAN residual added to a *plain-formula* Black-Scholes anchor (no regime conditioning on the anchor).
- `spikan_plain_iv_hybrid`: Plain-formula anchor with a learned *IV-space* residual. This is the headline hybrid: it corrects the implied volatility surface rather than the price directly.

Evaluation metrics. Protocol tiers: execution rate, intent-match rate, DSL validity rate, semantic signature match, group consistency. Pricing tiers: price MAE by regime, IV MAE on the live surface. Engineering: throughput (smp/sec), queue overflow count, GPU blocking time (ms).

Note on sample sizes. The protocol benchmarks involve 15–23 prompts and 4 paraphrase groups. At these sizes, individual results (especially group-level pass rates such as $3/4 = 0.75$) should be interpreted as directional estimates rather than precise measurements. We do not report confidence intervals because bootstrapping 23 binary outcomes produces very wide intervals that would obscure rather than illuminate the findings. The appropriate response to this limitation is to increase benchmark scale in future work, not to over-interpret the current point estimates.

7.3 Protocol Benchmark Tiers

Tier 0A (Curated benchmark): 15 prompts (9 direct, 6 narrative) spanning `OUTLOOK`, `QUOTE`, `SURFACE`, and `SCENARIO` operations, written in the style of the system prompt and few-shot examples. This tier establishes the *floor of protocol viability* — it does not measure generalization to out-of-distribution prompts.

Tier 0B (Document-grounded benchmark): 8 prompts derived from real local document excerpts (fluid dynamics notes, COMEX inventory summaries, regime-shift research notes) with analyst requests appended. This tier tests robustness to longer, noisier context and is the primary evidence tier for deployment planning.

Tier 0C (Semantic stability benchmark): Paraphrase groups, each containing multiple phrasings of the same analyst intent. Tests whether the DSL action is invariant to surface variation.

Tier 0D (Regime-action coherence benchmark): Holds the query family fixed and varies only the DSL regime instruction (`STABLE`, `TRANSITION`, `STRESS`). The five financial monotonicity checks test whether the backend responds with directionally ordered outputs under each regime context.

Tier 0E (Document-to-gold agreement): Human-authored gold DSL commands are specified in advance for each document-grounded prompt. Agent output is compared at three levels: exact normalized string equality, semantic signature equality, and executed-result equality. This is the strictest tier and the primary measure of distance from full analyst automation.

8 Results and Findings

8.1 RQ1 – Protocol Validity

Table 1: Protocol results by context tier. Execute = fraction producing valid, executable DSL. Intent = fraction matching the analyst's stated goal. Tier 0A is a floor check (prompts written in-distribution with the system prompt); Tier 0B is the primary evidence tier (derived from real document excerpts).

Context	Cases	Execute	Intent	Mean latency (s)
Direct prompt (0A)	9	1.00	1.00	0.776
Narrative prompt (0A)	6	1.00	1.00	0.573
Document-grounded (0B)	8	1.00	1.00	2.747
Overall (0A)	**15**	**1.00**	**1.00**	**0.695**

All 15 curated and all 8 document-grounded prompts produce valid, executable DSL. The Tier 0A result (1.00 on in-distribution prompts) establishes viability but not generalization. The Tier 0B result — maintaining 1.00 intent-match on prompts derived from documents the system prompt was not designed around — is the more meaningful finding.

The 4× latency increase from direct (0.78 s) to document-grounded (2.75 s) reflects a genuine deployment tradeoff: document-grounded latency is acceptable for the slow strategic path (System 2, 100-step period) but not for the fast reflexive path (System 1, must stay under 50 ms).

Representative compiled outputs from Tier 0B:

- Fluid-dynamics excerpt → `OUTLOOK asset=silver horizon=10d backdrop=STRESS` (shock-risk score 13.45)
- COMEX inventory note → `QUOTE call spot=31 strike=32 expiry=45d vol=kan_regime backdrop=STRESS rate=5%`
- Regime-shift challenge → `SURFACE vol asset=silver regime=STRESS strikes=[0.9,1.0,1.1] T=30d`

8.2 RQ2 – Paraphrase Stability

Three of four paraphrase groups fully preserve both the DSL action and the executed outcome across all phrasings. The failing group (`transition_surface`) exposes a specific normalization ambiguity: one paraphrase caused the agent to expand a three-point strike grid to five points while correctly identifying the regime and operation. This is a normalization failure (different structural choice) rather than a comprehension failure (wrong intent).

Table 2: Semantic stability benchmark (Tier 0C). "Group" metrics require all paraphrases within a group to produce the same DSL action and execution result. $n = 4$ paraphrase groups; interpret the 0.75 group pass rate as a directional estimate from a small sample.

Metric	Value
Case translation success	1.00
Case valid DSL	1.00
Case execution success	1.00
Case semantic signature match	0.9167
Group signature consistency	0.75
Group result consistency	0.75
All-group pass rate	0.75

The 0.75 group pass rate — noting the small $n = 4$ caveat above — suggests that one in four equivalent analyst queries may produce a behaviorally different but intent-preserving command. Closing this gap is a system-prompt engineering problem (Section 10), not an architectural one.

8.3 RQ3 – Normalization Gap

Table 3: Document-to-gold agreement (Tier 0E; $n = 8$ prompts). "Exact" = normalized string equality with the human-authored gold DSL. "Semantic" = same DSL operation family and regime. "Result" = same executed numerical output.

Metric	Value
Translation success	1.00
Exact DSL agreement	0.375
Valid DSL	1.00
Semantic agreement	0.75
Execution success	1.00
Result agreement	0.75

The gap between exact agreement (0.375) and semantic/result agreement (0.75) is the central diagnostic signal of the evaluation. The system reliably produces valid, semantically aligned DSL but does not reliably produce the *exact* form a human analyst would have chosen. Both failure modes are interpretable: *regime under-escalation* (STRESS compiled as TRANSITION) and *surface grid broadening* (three-point grid expanded to five). Neither represents a semantic misunderstanding; both represent different but valid structural choices. The path to improvement is contract tightening on the system prompt, not architectural change.

8.4 RQ4 – Regime Coherence

To test regime coherence, we define two Heston parameter sets that represent canonical market conditions. The **STABLE** anchor uses (κ=3.0, θ=0.02, σ=0.15, ρ=−0.50), representing a calm, mean-reverting market with low vol-of-vol. The **STRESS** anchor uses (κ=0.5, θ=0.09, σ=0.60, ρ=−0.85), representing a crash-risk environment with slow mean-reversion and high vol-of-vol. Both sets use r=0.05, K=100, T=1. We query each model at 500 uniformly sampled (S, v, t) points and check five monotonicity conditions.

This result requires careful interpretation. The raw model failing all five checks does not mean the model is wrong; it means PDE residual minimization is an *insufficient* training objective for producing regime-ordered behavior when queried at out-of-distribution parameter sets. The

Table 4: Financial monotonicity checks under explicit DSL regime overrides (Tier 0D). "Raw" = model queried directly under the STABLE/STRESS Heston parameter anchors. "With Semantic Controller" = explicit controller with monotone projection applied over raw outputs. All five checks fail for the raw model; all five pass with the controller.

Monotonicity Check	Raw Output	With Semantic Controller
Quote implied vol: STABLE → STRESS	×	✓
Quote price: STABLE → STRESS	×	✓
Surface mean vol: STABLE → STRESS	×	✓
Outlook shock-score: STABLE → STRESS	×	✓
Shock-risk labels ordered low to high	×	✓

model correctly minimizes what it was trained on. The key architectural insight is that *monotonicity is currently a property of the control layer, not of the learned weights.* Whether monotonicity should be built into the training objective (via ranking losses) or the architecture (via MonoKAN substitution) is the most direct open research question this result raises.

8.5 RQ5 – Pricing Performance

Table 5: Replay price MAE by market regime. Lower is better. `spikan_plain_iv_hybrid` adds a learned IV-space residual to a plain Black-Scholes anchor (no regime conditioning on the anchor); `spikan_plain_hybrid` adds a price-space residual to the same anchor. See Section 7 for full model definitions.

Model	Stable	Transition	Stress	Overall
`bs_plain`	0.8398	2.1037	1.6355	1.4597
`spikan_plain_hybrid`	1.1542	2.4827	0.4745	1.3705
`spikan_plain_iv_hybrid`	1.1628	1.4622	0.4239	**0.9317**
`rf_direct`	8.3183	10.2679	1.8707	6.8190

Table 6: Live term-surface results (one quote date, cross-sectional split; see Limitation 1). IV MAE measures the ability to reconstruct the observed implied volatility surface.

Model	Price MAE	IV MAE
`spikan_plain_hybrid`	0.8976	0.1578
`rf_direct`	1.0444	**0.0781**
`bs_plain`	1.4646	0.1601
`spikan_hybrid`	1.6793	0.1863
`bs_regime`	12.6503	0.8314

The IV-residual hybrid achieves the best overall replay MAE (0.9317), with particular strength in transition (1.46) and stress (0.42) — the conditions where regime-conditioned corrections matter most. In stable conditions, plain Black-Scholes (0.84) is the best single model, consistent with the formula being most accurate when market conditions match its assumptions. The random-forest baseline is the best IV reconstructor (IV MAE 0.0781) but the worst replay pricer overall (6.82) — the canonical pattern of a model that minimizes in-distribution reconstruction error but generalizes poorly under distributional shift.

The cross-asset results confirm the architecture's domain specificity: the hybrid wins on silver and oil (assets where macro narrative and supply-shock interpretation are operationally rele-

Table 7: Cross-asset pricing summary. The best model family and its price MAE vary substantially across assets, reflecting different regime signal relevance. See text for interpretation.

Asset	Dominant Signal Type	Best Model	Price MAE
SLV (silver)	Macro / supply narrative	`spikan_plain_iv_hybrid`	0.9317
GLD (gold)	Mean-reverting vol structure	`bs_plain`	1.6515
USO (oil)	Inventory shock / macro	`spikan_plain_iv_hybrid`	0.4897
SOXX (semiconductors)	Structural / earnings	`rf_direct`	1.7194
XLK (technology)	Structural / earnings	`rf_direct`	2.5180

vant), while gold, semiconductor, and technology equities favor simpler baselines. For gold, the mean-reverting vol structure is well-captured by the Black-Scholes formula. For SOXX and XLK, the relevant pricing signals (earnings surprise, export controls, AI capital expenditure) are absent from the current feature set. This is a signal about where feature engineering investment is needed, not a failure of the architecture.

8.6 RQ6 – Engineering Performance

Table 8: Engineering performance on NVIDIA H200, batch size 4,096. Training throughput (26,300 smp/sec) is measured during sustained 5-epoch training of 1M rows; the standalone `validate_e2e.py` microbenchmark reports 6,506 smp/sec on a 50-step cold-start run with a lighter synthetic dataset. Both configurations use identical hardware and batch size; the difference reflects startup amortization, not a measurement error.

Metric	Value
Sustained training throughput (5 epochs, 1M rows)	26,300 smp/sec
E2E microbenchmark throughput (50 steps, cold start)	6,506 smp/sec
System 1 (fast path) latency	<50 ms
System 2 (slow path) latency	200–500 ms
Decision queue overflows (1M-row stress test)	0
GPU blocking time from LLM	0 ms
Dual-process vs. baseline throughput delta	+4.4%

The asynchronous dual-queue architecture maintains full GPU throughput with zero queue overflows and zero GPU blocking. The dual-process configuration shows +4.4% higher effective throughput than a pure gradient-descent baseline in the 1M-row stress test. A likely contributor is System 1's progressive pruning of near-zero edges, which reduces the computational graph size over time. However, we do not have an ablation isolating this effect (e.g., pruning count vs. throughput trajectory), so this remains a hypothesis for future investigation rather than a confirmed causal claim.

In a hypothetical synchronous design, a 500 ms System 2 decision would block $\approx$ 3,200 training steps at sustained throughput. The asynchronous design reduces this cost to zero.

9 Discussion

9.1 What the Architecture Gets Right

Three claims receive preliminary support.

The semantic compiler interface is viable on Tier 0B inputs. All 8 document-grounded

prompts (derived from documents not used in constructing the system prompt) produce valid, executable, intent-matching DSL. This is the meaningful evidence threshold: it shows the protocol layer survives context not of its own design.

Separation of concerns makes the system debuggable. Because each layer is evaluated independently, failures are localizable: a protocol failure does not implicate the PDE solver; a monotonicity failure does not implicate the LLM compiler. This debuggability is a structural property of the architecture, not an empirical result.

The IV-residual hybrid is the right pricing architecture for the target use case. The regime-by-regime MAE profile (best in transition and stress, weaker in stable) is exactly the pattern the architecture predicts: the learned correction adds value when conditions deviate from the analytic formula's assumptions. This is a structural argument, not just a post-hoc observation.

9.2 The Monotonicity Gap: A Training Objective Problem

The most important finding is that PDE residual minimization is not a sufficient training objective for regime-ordered pricing. The model is not wrong; it minimizes what it was trained on. The architectural question this raises is whether the training objective can be enriched with regime-ordering terms — e.g., ranking losses on pairs of STABLE/STRESS outputs — such that monotonicity becomes intrinsic rather than requiring an external controller. This is the primary open design question.

9.3 The Normalization Gap: A Contract Problem

The gap between exact DSL agreement (0.375) and semantic agreement (0.75) on Tier 0E is structurally similar to a software calling-convention problem: the compiler is correct, but the contract is underspecified. Both failure modes (regime under-escalation and grid broadening) can be addressed by tighter normalization rules in the system prompt without architectural change.

9.4 Cross-Asset Transfer: What Domain Specificity Means

The cross-asset heterogeneity is not a failure of the architecture — it is a signal about domain fit. The architecture provides value where narrative regime interpretation is part of the pricing problem (silver, oil). It provides no value where the pricing signal is structural and not in the current feature set (semiconductor earnings, technology capex). This tells us precisely which feature engineering is needed to extend the architecture's advantage to equity derivatives.

10 Limitations and Future Directions

We state the following as definite scope boundaries, each paired with the specific next step that would address it.

1. **Single quote date for the live benchmark.** The live term-surface benchmark uses one quote date with a cross-sectional split — a sanity check, not a generalization result. *Next step*: obtain multi-date historical OPRA tapes (or the public CBOE historical options dataset) and repeat the replay benchmark over a rolling 12-month window.

2. **Structural replay, not real historical options.** The replay benchmark uses a replayed SLV surface template, not real historical quotes. Stress-regime MAE values should be interpreted as directional evidence, not production-ready estimates. *Next step*: acquire a real multi-date historical options chain for the same replay period.

3. **Curated and near-distribution protocol benchmarks.** The 15 Tier 0A prompts were written in-distribution with the system prompt; they establish viability, not adversarial robustness. Tier 0B provides stronger evidence but is still curated. *Next step*: construct a benchmark drawn from raw financial news feeds with adversarial phrasings.

4. **Small benchmark sample sizes.** With $n \leq 15$ prompts per tier, group-level pass rates (e.g., $3/4 = 0.75$) are estimated from very few data points. *Next step*: scale the benchmark to ≥ 100 prompts per tier before drawing deployment conclusions.

5. **Exact DSL agreement ceiling of 0.375.** The remaining normalization failures on Tier 0E are interpretable but real. *Next step*: add a monotone escalation rule and canonical grid specification to the system prompt; run ablations on Tier 0E to establish the ceiling achievable without fine-tuning.

6. **Monotonicity is extrinsic.** All five monotonicity checks pass because of the semantic controller, not the raw model. *Next step (a)*: add regime-ordering ranking losses to the training objective — sample pairs (S, v, t) under STABLE and STRESS parameter sets and penalize $\max(0, V_{\text{STABLE}} - V_{\text{STRESS}} + \epsilon)$. *Next step (b)*: selectively replace B-spline edges with MonoKAN-style certified monotone variants [Polo-Molina et al., 2024] on the vol-surface path where the controller correction is heaviest.

7. **Domain-limited cross-asset advantage.** The hybrid does not universally outperform simpler baselines. *Next step*: for SOXX and XLK, add earnings surprise indicators, export-control event flags, and AI capital expenditure indices as HMM input features, then repeat the cross-asset experiments.

8. **Security analysis is proof-of-concept.** The AST allowlist validator blocks all five injection payloads tested but has not been subjected to formal red-teaming. *Next step*: engage a security review against a broader adversarial payload suite; additionally add runtime differentiability probing on a sample grid.

9. **No fine-tuned model weights.** The `LoraDataCollector` infrastructure exists for collecting LiuClaw agent trajectories. *Next step*: once sufficient production trajectory data has been collected, apply LoRA [Hu et al., 2021] fine-tuning to the 7B model (System 1 reflexive task) and the 32B model (System 2 strategic task) to reduce latency and close the normalization gap.

10. **Retrieval and adversarial defense for the document input path.** Moving from curated excerpts to a live news feed requires a retrieval pipeline and input sanitization separate from the AST execution-side validator. *Next step*: design a document ingestion layer with deduplication and retrieval-level injection defense.

11 Conclusion

We have presented the semantic compiler as a design pattern for safe, auditable LLM-driven topology management of physics-constrained neural networks, and demonstrated it in the Op-KAN system — a three-layer architecture for real-time options pricing under the Heston stochastic volatility model.

The pattern's three enforcement layers — Pydantic type validation, AST allowlist analysis, and atomic mutation rollback — together provide a boundary at which every LLM-driven network modification is type-safe, vocabulary-restricted, and transactionally reversible. The dual-process agent (System 1 reflexive, System 2 strategic) ensures neither LLM path blocks the GPU training loop.

The most consequential finding is not about pricing accuracy: it is that PDE residual minimization is *insufficient* as a training objective for regime-ordered behavior. The raw model correctly minimizes what it was trained on; regime ordering requires the semantic controller. This finding generalizes beyond options pricing to any physics-constrained network deployed under distributional shift: the physics prior constrains the solution on the training domain, but does not enforce behavioral ordering at out-of-distribution query points. Enriching the training objective with regime-ordering ranking losses is the most direct path to making this property intrinsic.

The semantic compiler boundary — natural language understanding in the LLM, mathematical constraint satisfaction in the physics-informed model, auditability in the typed DSL — is the architectural contribution this paper makes. Each layer does one thing well and does not ask the other layers to compensate. The next phase is making each layer individually stronger within those boundaries: the learned model through ranking losses, the agent through domain fine-tuning on collected trajectories, the vocabulary layer through runtime differentiability probing. The boundaries remain; the capabilities deepen.

References

Hans Bühler, Lukas Gonon, Josef Teichmann, and Ben Wood. Deep hedging. *Quantitative Finance*, 19(8):1271–1291, 2019.

Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.

James D Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.

Steven L Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *Review of Financial Studies*, 6(2):327–343, 1993.

Blanka Horvath, Aitor Muguruza, and Mehdi Tomas. Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models. volume 21, pages 11–27, 2021.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Allen H Huang, Hui Wang, and Yi Yang. FinBERT: A large language model for extracting information from financial text. *Contemporary Accounting Research*, 40(2):806–841, 2023.

Daniel Kahneman. *Thinking, Fast and Slow.* Farrar, Straus and Giroux, 2011.

Patrick Kidger, James Foster, Xuechen Li, and Terry Lyons. Neural SDEs as infinite-dimensional GANs. In *International Conference on Machine Learning*, pages 5453–5463. PMLR, 2021.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

Andrei Nikolaevich Kolmogorov. On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. In *Doklady Akademii Nauk SSSR*, volume 114, pages 953–956. 1957.

Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations*, 2019.

Ziming Liu, Yixuan Wang, Vaibhav Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y Hou, and Max Tegmark. KAN: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.

Alejandro Polo-Molina, David Alfaya, and Javier Portela. MonoKAN: Certified monotonic kolmogorov-arnold network. *arXiv preprint arXiv:2409.11078*, 2024.

Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessè, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Paul Wilmott, Jeff Dewynne, and Sam Howison. *Option Pricing: Mathematical Models and Computation*. Oxford Financial Press, 1993.

Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. BloombergGPT: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.